

4. Erőforrásigény. A kontrollált pont-összeadás dominanciája miatt az összes kapuszám $O(n^3)$:

- Moduláris szorzás: $O(n^2)$ Toffoli kapu (Montgomery vagy iskolai szorzás)
- Moduláris inverz (kiterjesztett Euklideszi alg.): $O(n^2)$ Toffoli kapu
- Egy pont-összeadás: 2 inverz + 3 szorzás $\rightarrow O(n^2)$ Toffoli
- Skalár-szorzás ($2n$ pont-összeadás): $O(n^3)$ Toffoli összesen

A pontos konstansok a különböző implementációkban eltérnek – ez eredményezi az 7. ábrán látható erőforrás-bebecslések szóródását 2004-től 2026-ig.

11.2 Grover algoritmus

Míg Shor algoritmus a struktúrált problémákat (prímtényezős felbontás, DLP) támadja exponenciális gyorsítással, Grover algoritmus [97] egy általános keresési feladatot old meg négyzetgyök-gyorsítással. Ez kevésbé drámai, de univerzálisan alkalmazható minden szimmetrikus kulcsú és hash-alapú rendszerre.

11.2.1 A probléma

Adott egy $f : \{0, 1\}^n \rightarrow \{0, 1\}$ orákulum-függvény (fekete doboz), amelyre pontosan egy x_0 bemenet esetén $f(x_0) = 1$ (a keresett elem), és minden más x -re $f(x) = 0$. A feladat x_0 megtalálása. Klasszikusan ehhez átlagosan $N/2 = 2^{n-1}$ lekérdezés szükséges (ahol $N = 2^n$). Grover algoritmus $O(\sqrt{N})$ lekérdezéssel megoldja.

Ez a strukturálatlan keresés a hash-függvények két klasszikus támadási feladatának absztrakt megfelelője. Érdekes ezeket külön tisztázni, mert a kvantumgyorsítás mértéke – és így a szükséges védekezés – a kettő esetében eltérő.

Preimage-probléma (előképkeresés). Adott egy H hash-függvény és egy cél y érték; keressünk olyan x bemenetet, amelyre $H(x) = y$. Ez pontosan a fenti keresési feladat: a „jó” bemenetek halmaza a $\{x : H(x) = y\}$, amit egy 2^n méretű térben keressünk. Klasszikusan átlagosan 2^n (illetve 2^{n-1}) próbálkozás kell, kvantumosan Grover algoritmusával $O(2^{n/2})$. Ide tartozik a második előképkeresés (second preimage) is: adott x_1 mellett keressünk egy másik $x_2 \neq x_1$ bemenetet ugyanazzal a hash-értékkel; ennek nehézsége lényegében megegyezik a preimage-ével.

Collision-probléma (ütközéskeresés). Keressünk *tetszőleges* két különböző bemenetet, $x_1 \neq x_2$, amelyekre $H(x_1) = H(x_2)$. A preimage-nél egy előre megadott y hash-értékhez kellett bemenetet találni; itt viszont nincs ilyen előre kitűzött célérték: elég *bármely* két olyan bemenetet megtalálni, amelynek egyezik a hash-e. Éppen ezért a feladat lényegesen könnyebb, mint a preimage. Klasszikusan a **születésnap-paradoxon** (birthday attack) miatt már $\approx 2^{n/2}$ értékelés után nagy valószínűséggel találunk ütközést – innen a név: ahogy már 23 ember között is $> 50\%$ eséllyel van két azonos születésnapú, úgy egy 2^n méretű kimeneti térben is elég $\sqrt{2^n} = 2^{n/2}$ minta az ütközéshez. Kvantumosan a legjobb ismert algoritmus (BHT, Brassard-Høyer-Tapp [98]) $O(2^{n/3})$ értékeléssel oldja meg.

Összegzés. Az alábbi táblázat a jelenlegi leggyorsabb ismert algoritmusok költségét foglalja össze egy n bites kimenetű hash-függvényre (a nagyságrendek a lekérdezések/értékelések számát jelölik):

Feladat	Klasszikus	Kvantum
Preimage (előképkeresés)	$O(2^n)$	$O(2^{n/2})$ (Grover)
Second preimage	$O(2^n)$	$O(2^{n/2})$ (Grover)
Collision (ütközéskeresés)	$O(2^{n/2})$ (birthday)	$O(2^{n/3})$ (BHT)

Ez magyarázza, miért kell a kvantumkorban a hash-kimenet hosszát megnövelni: egy 128 bites biztonsági szint eléréséhez a preimage-ellenállás miatt 256 bites hash-kimenet kell (Grover $2^{256/2} = 2^{128}$), az ütközésállóság pedig ugyanígy 256 bites kimenetnél klasszikusan 2^{128} , kvantumosan 2^{85} szinten áll – a gyakorlatban ezért a hash-kimenet megduplázása a bevett védekezés.

11.2.2 Az algoritmus

1. **Inicializálás:** Készítsünk egyenletes szuperpozíciót:

$$|\psi_0\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle$$

2. **Grover-iteráció:** Ismételjük az alábbi két lépést $\approx \frac{\pi}{4}\sqrt{N}$ -szer:

(a) *Orákulum (fázis-invertálás):* Az orákulum megjelöli a keresett elemet fázisfor-
gatással:

$$O_f : |x\rangle \mapsto (-1)^{f(x)}|x\rangle$$

Ez csak $|x_0\rangle$ fázisát invertálja, a többi állapotot változatlanul hagyja.

(b) *Diffúziós operátor (átlag körüli tükrözés):*

$$D = 2|\psi_0\rangle\langle\psi_0| - I$$

ahol I az identitás. Ez az operátor minden amplitúdót az átlag körül tükröz, ezáltal felerősíti a megjelölt elem amplitúdóját.

3. **Mérés:** A $\frac{\pi}{4}\sqrt{N}$ iteráció után mérjük meg a regisztert. Az eredmény $\Omega(1)$ valószínűséggel x_0 .

11.2.3 Geometriai interpretáció

Az algoritmus működése szemléletesen megérthető egy kétdimenziós síkban. Jelöljük:

$$|w\rangle = |x_0\rangle, \quad |s'\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq x_0} |x\rangle$$

Az aktuális állapot mindig a $\{|w\rangle, |s'\rangle\}$ sík egy vektora. Kezdetben az állapot közel áll $|s'\rangle$ -hoz (szöge $\theta/2$, ahol $\sin(\theta/2) = 1/\sqrt{N}$). Minden Grover-iteráció θ szöggel forgatja az állapotvektort $|w\rangle$ felé. Tehát $\pi/(2\theta) \approx \frac{\pi}{4}\sqrt{N}$ iteráció után az állapot közel kerül $|w\rangle$ -hez.

11.2.4 Kriptográfiai következmények

- **Szimmetrikus kulcsú titkosítás:** Egy n bites kulcs brute-force keresése $O(2^{n/2})$ -re csökken. Ezért az AES-128 posztkvantum biztonsága csak 64 bit $\rightarrow$ AES-256 szükséges a 128 bites kvantumbiztonsághoz.
- **Hash-függvények:** A preimage keresés $O(2^{n/2})$ -re gyorsul, ezért 256 bites hash-kimenet szükséges 128 bites posztkvantum biztonsághoz.
- **Korlátok:** Grover algoritmus nem párhuzamosítható hatékonyan: p párhuzamos kvantumszámítógéppel a gyorsítás csak $O(\sqrt{N/p})$, ami lényegesen gyengébb mint a klasszikus párhuzamosítás. Emiatt a valós fenyegetés a szimmetrikus rendszerekre mérsékelt – a kulcsméret megduplázása elegendő kompenzáció.

Daniel J. Bernstein a posztkvantum kriptográfiáról szóló könyvének bevezetőjében [96] a következő gondolat kísérlettel érzékelteti a helyzetet: képzeljük el, hogy holnap megépül egy nagy kvantumszámítógép, és a The New York Times címlapján jelenik meg a hír, hogy az internet védelmére használt összes nyilvános kulcsú algoritmus feltörésre került – a felhasználók pánikba esnek. Bernstein rámutat, hogy ez a pánik túlzás lenne: az RSA, DSA és ECDSA megsemmisítéséből nem következik, hogy a kriptográfia halott. Léteznek fontos kriptográfiai sémacsaládok – hash-alapú, kódalapú, rácslapú és többváltozós kriptográfia –, amelyekre eddig senki sem tudta alkalmazni Shor algoritmusát. Grover algoritmus [97] ugyan bizonyos hatással van ezekre a rendszerekre is, de korántsem olyan drámai, mint Shor; a kriptográfusok egyszerűen valamivel nagyobb kulcsméretekkal kompenzálhatják.

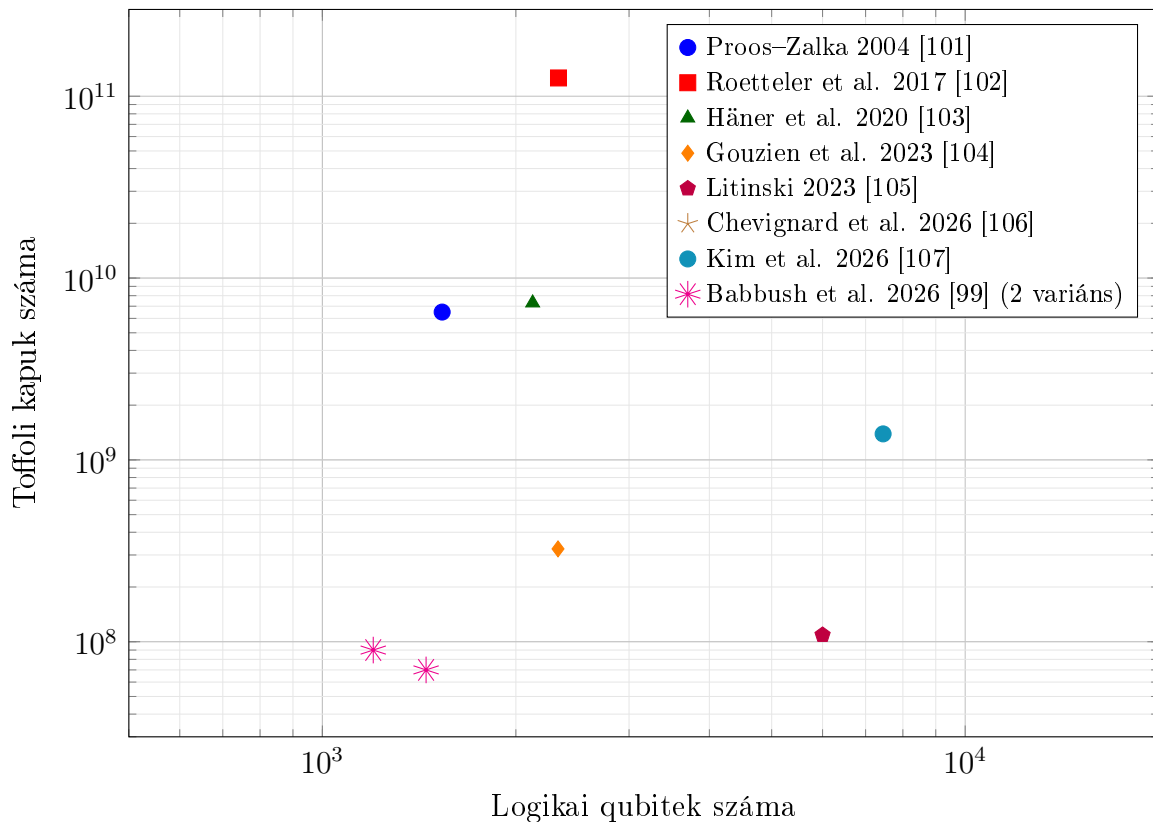
A fenyegetés nem pusztán elméleti. A kvantumhardver fejlődése évről évre gyorsul: míg 2019-ben a Google 53 qubites Sycamore processzora demonstrálta a kvantumfölényt egy speciális feladaton [110], addig 2024–2025-re az IBM, a Google és kínai kutatócsoportok már az 1000 fizikai qubites határt közelítik [111, 112]. A kriptográfiailag releváns kvantumszámítógép (CRQC – Cryptographically Relevant Quantum Computer) megjelenése ugyan még évtizedekre lehet, de a „harvest now, decrypt later” támadási modell – amelyben egy támadó ma rögzíti a titkosított forgalmat, hogy majd a jövőben kvantumszámítógéppel visszafejtse – már most valós kockázatot jelent a hosszú élettartamú titkokra nézve. Hallgren és Vollmer kiemelik [96], hogy a ma legérzékenyebb titkosított adatokat már most kvantumrezisztens algoritmusokkal kell védeni, mert ezeknek az adatoknak még a kvantumszámítógépek megépítése után is biztonságban kell maradniuk.

A kvantumszámítások erőforrásigényét gyakran a **Toffoli kapu** (más néven CCNOT – controlled-controlled-NOT) számában mérik [108]. A Toffoli kapu egy háromqubites reverzibilis logikai kapu, amely két vezérlő qubit együttes $|1\rangle$ állapota esetén invertálja a célqubitet: $|a, b, c\rangle \mapsto |a, b, c \oplus (a \cdot b)\rangle$. A hibátűrő kvantumszámításban a Toffoli kapu azért kulcsfontosságú mérőszám, mert a felületi kódokon (surface code) való implementálása költséges: ún. mágikus állapotok (magic states) desztillálását igényli, szemben a Clifford kapukkal, amelyek tranzverzálisan végrehajthatók [109]. Egy kvantumalgoritmus „nehézsége” tehát jól jellemezhető a szükséges logikai qubitek és Toffoli kapuk számával.

A legfrissebb eredmények számszerűsítik a fenyegetést. A Google Quantum AI kutatócsoportja 2026-ban publikálta [99], hogy a 256 bites elliptikus görbe diszkrét logaritmus probléma (ECDLP) a secp256k1 görbén – amelyet a Bitcoin, Ethereum és számos más blokklánc használ – Shor algoritmusával megoldható mindössze ≤ 1200 logikai qubittel és ≤ 90 millió Toffoli kapuval (vagy alternatívan ≤ 1450 logikai qubittel és ≤ 70 millió Toffoli

kapuval). Szupravezető architektúrán, 10^{-3} fizikai hibaráta és planáris összeköttetés mellett ez kevesebb mint félmillió fizikai qubittel, mindössze percek alatt végrehajtható. A szerzők megkülönböztetik a „gyors órajelű” (szupravezető, fotonikus) és a „lassú órajelű” (semleges atom, ioncsapda) architektúrákat: az előbbiek képesek lennének „on-spend” támadásra, azaz egy tranzakció elfogásával a privát kulcs kiszámolására még azelőtt, hogy a tranzakció bekerülne a blokkba (kb. 9 perc alatt). A Bitcoin P2PK (Pay-to-Public-Key) szkriptjei több mint 1,7 millió BTC-t védenek közvetlenül kitett publikus kulccsal, a teljes kvantumsebezhető mennyiség elérheti a 2,3 millió BTC-t. Az Ethereum esetében a top 1000 leggazdagabb számla (együttesen kb. 20,5 millió ETH) kevesebb mint 9 nap alatt feltörhető lenne egy gyors órajelű CRQC-vel.

Az 7. ábra összefoglalja a 256 bites ECDLP feltöréséhez szükséges logikai kvantum-erőforrások fejlődését az elmúlt két évtizedben.



7. ábra: A 256 bites ECDLP (secp256k1 / P-256) kvantummal történő feltöréséhez szükséges logikai erőforrások összehasonlítása. A vízszintes tengely a szükséges logikai qubitek számát, a függőleges tengely a Toffoli (vagy T-) kapuk számát mutatja logaritmusos skálán. Az ábra jól szemlélteti az erőforrásigény drámai csökkenését 2004 és 2026 között: míg a korai becslések 10^{11} kapuval és több ezer qubittel számoltak, addig a Google Quantum AI 2026-os eredménye mindössze ~ 1200 qubittel és $< 10^8$ Toffoli kapuval oldja meg a problémát.

Emiatt az elmúlt évtizedben intenzív kutatás indult olyan kriptográfiai primitívek kidolgozására, amelyek ellenállnak mind a klasszikus, mind a kvantumtámadásoknak. A posztkvantum algoritmusok biztonsága más típusú matematikai problémákra épül, mint a klasszikus sémáké:

- **Rácsalapú (lattice-based) kriptográfia:** a legrövidebb vektor probléma (SVP)

és a Learning With Errors (LWE) probléma nehézségén alapul. Ide tartozik az ML-KEM és az ML-DSA.

- **Hash-alapú aláírások:** kizárólag a használt hash-függvény biztonságára épülnek, minimális matematikai feltételezéssel. Ide tartozik az SLH-DSA (SPHINCS+) és a XMSS.
- **Kódalapú (code-based) kriptográfia:** hibajavító kódok dekódolásának nehézségét használja ki. McEliece (1978) az egyik legrégebbi ilyen séma, és a mai napig nem sikerült hatékonyan megtörni.
- **Többváltozós polinomrendszerek:** nemlineáris egyenletrendszerek megoldásának nehézségén alapul.

11.3 ML-KEM (Module-Lattice-Based Key Encapsulation Mechanism)

Az ML-KEM (korábban CRYSTALS-Kyber) a NIST által 2024-ben szabványosított poszt-kvantum kulscserélő mechanizmus (FIPS 203) [113]. Biztonsága a Module Learning With Errors (MLWE) probléma nehézségén alapul. Az alábbiakban az ML-KEM működését mutatjuk be az egyenletek szintjén.

11.3.1 Alapfogalmak

Legyen q egy prím (ML-KEM-ben $q = 3329$), $n = 256$ a polinom foka, és $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ a polinom-gyűrű. Az ML-KEM $k \times k$ méretű mátrixokkal dolgozik az R_q gyűrű felett, ahol $k \in \{2, 3, 4\}$ a biztonsági szinttől függően (ML-KEM-512, ML-KEM-768, ML-KEM-1024).

Jelöljük β_η -val azt az eloszlást, amely az R_q elemeit úgy generálja, hogy minden együttható egy η paraméterű centrált binomiális eloszlásból származik.

11.3.2 Kulcsgenerálás (KeyGen)

1. Generáljunk egy egyenletes eloszlású mátrixot $\mathbf{A} \in R_q^{k \times k}$ egy véletlenszerű seedből.
2. Mintavételezzünk kis normájú titkos vektort $\mathbf{s} \leftarrow \beta_\eta^k$ és hibavektort $\mathbf{e} \leftarrow \beta_\eta^k$.
3. Számoljuk ki a publikus kulcsot:

$$\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \in R_q^k$$

A publikus kulcs: $(\mathbf{A}, \mathbf{t})$. A titkos kulcs: $\mathbf{s}$.

11.3.3 Enkapszuláció (Encaps)

Az enkapszuláció során a küldő a címzett publikus kulcsával egy közös titkot hoz létre, és annak titkosított formáját (ciphertextet) küldi el. Erre azért van szükség, hogy a két fél biztonságos csatorna nélkül is egyforma szimmetrikus kulcshoz jusson, amelyet ezután a tényleges kommunikáció titkosítására használhatnak.

Adott a publikus kulcs $(\mathbf{A}, \mathbf{t})$ és egy véletlenszerű üzenet $m \in \{0, 1\}^{256}$:

1. Mintavételezzünk véletlen vektorokat: $\mathbf{r} \leftarrow \beta_\eta^k$, $\mathbf{e}_1 \leftarrow \beta_\eta^k$, $e_2 \leftarrow \beta_\eta$.
2. Számoljuk ki a ciphertext két komponensét:

$$\mathbf{u} = \mathbf{A}^\top \cdot \mathbf{r} + \mathbf{e}_1 \in R_q^k$$

$$v = \mathbf{t}^\top \cdot \mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rfloor \cdot m \in R_q$$

A ciphertext: $(\mathbf{u}, v)$. A tényleges megosztott titkot (vagyis a szimmetrikus kulcsot) nem közvetlenül az m üzenet adja, hanem az m -re alkalmazott kriptográfiai hash-függvény kimenete, azaz $K = H(m)$. Erre azért van szükség, mert a hash-függvény az esetleges strukturális szabályosságokat eltünteti, és egyenletes eloszlású, rögzített hosszúságú kulcsot állít elő, amely biztonságosan használható szimmetrikus titkosításhoz.

11.3.4 Dekapszuláció (Decaps)

A titkos kulcs $\mathbf{s}$ birtokában a címzett visszafejti m -et:

$$\begin{aligned} v - \mathbf{s}^\top \cdot \mathbf{u} &= \mathbf{t}^\top \mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rfloor m - \mathbf{s}^\top (\mathbf{A}^\top \mathbf{r} + \mathbf{e}_1) \\ &= (\mathbf{A}\mathbf{s} + \mathbf{e})^\top \mathbf{r} + e_2 + \left\lfloor \frac{q}{2} \right\rfloor m - \mathbf{s}^\top \mathbf{A}^\top \mathbf{r} - \mathbf{s}^\top \mathbf{e}_1 \\ &= \mathbf{e}^\top \mathbf{r} + e_2 - \mathbf{s}^\top \mathbf{e}_1 + \left\lfloor \frac{q}{2} \right\rfloor m \end{aligned}$$

Mivel $\mathbf{e}$, $\mathbf{r}$, e_2 , $\mathbf{s}$ és $\mathbf{e}_1$ mind kis normájúak, a $\mathbf{e}^\top \mathbf{r} + e_2 - \mathbf{s}^\top \mathbf{e}_1$ hibatag abszolút értékben jóval kisebb mint $\lfloor q/4 \rfloor$. Ezért az m minden bite visszafejthető egyszerű küszöböléssel: ha az együttható közelebb van $\lfloor q/2 \rfloor$ -hoz, akkor $m_i = 1$, egyébként $m_i = 0$.

Érdeemes megérteni, miért szükséges a kis norma feltétel és mi történne, ha nem teljesülne. A visszafejtés lényege az, hogy az m üzenet minden bitjét a $\lfloor q/2 \rfloor \approx 1665$ értékkel kódoljuk (ha $m_i = 1$) vagy 0-val (ha $m_i = 0$). A visszafejtésnél a kapott együtthatóból kell eldönteni, hogy az eredeti bit 0 vagy 1 volt. A hibatag $(\mathbf{e}^\top \mathbf{r} + e_2 - \mathbf{s}^\top \mathbf{e}_1)$ zajként jelenik meg ezen értékek tetején. Ha ez a zaj kisebb mint $\lfloor q/4 \rfloor \approx 832$, akkor a küszöbölés egyértelműen működik, mert a 0 és a $\lfloor q/2 \rfloor$ közötti távolság $\lfloor q/2 \rfloor \approx 1665$, ennek fele pedig ≈ 832 , ami éppen a döntési határ. Amíg a zaj ennél kisebb, a helyes bit mindig kiolvasható.

Ha viszont $\mathbf{s}$ és $\mathbf{e}$ nem lennének kis normájúak (hanem például egyenletes eloszlásúak a teljes $\mathbb{Z}_q$ felett), a hibatag is egyenletesen oszlana el $\mathbb{Z}_q$ -ban. Ekkor a hibatag nagysága meghaladná a $\lfloor q/4 \rfloor$ küszöböt, és a küszöbölés véletlenszerű eredményt adna, vagyis a visszafejtés sikertelen lenne. Konkrétan az ML-KEM-768 paramétereivel ($q = 3329$, $k = 3$, $\eta = 2$, tehát az együtthatók a $\{-2, -1, 0, 1, 2\}$ halmazból származnak) a hibatag várható abszolút értéke $\sim k \cdot \eta^2 + \eta \approx 14$, ami bőven a 832-es határ alatt van. A visszafejtési hibaarány ezzel a paraméterezéssel kisebb mint 2^{-164} , tehát a gyakorlatban soha nem fordul elő hiba.

A rendszer biztonsága és helyessége tehát ellentétes követelményekre épül. Minél kisebb a zaj (kis η), annál megbízhatóbb a visszafejtés, de annál könnyebb a támadónak megkülönböztetni a publikus kulcsot egy véletlen mátrixtól (gyengébb biztonság). Minél nagyobb a zaj (nagy η), annál nehezebb az MLWE probléma (erősebb biztonság), de annál nagyobb a visszafejtési hiba valószínűsége. Az ML-KEM paraméterei ezt a kompromisszumot optimalizálják úgy, hogy mindkét követelmény teljesüljön egyszerre.

11.3.5 Biztonság

Az ML-KEM biztonsága azon a feltételezésen alapul, hogy a $(\mathbf{A}, \mathbf{As} + \mathbf{e})$ pár megkülönböztethetetlen egy egyenletes eloszlású $(\mathbf{A}, \mathbf{t})$ pártól, ha $\mathbf{s}$ és $\mathbf{e}$ kis normájúak. Ez az MLWE (Module Learning With Errors) probléma, amelyre sem klasszikus, sem kvantumalgoritmus nem ismert szubexponenciális futásidővel. A legjobb ismert klasszikus támadás a rácsredukció (pl. BKZ algoritmus), amelynek futásideje a rácsparaméterek megfelelő megválasztásával biztonságosan $> 2^{128}$ művelet fölé emelhető.

11.4 ML-DSA (Module-Lattice-Based Digital Signature Algorithm)

Az ML-DSA (korábban CRYSTALS-Dilithium) a NIST által 2024-ben szabványosított posztkvantum digitális aláírási séma (FIPS 204) [114]. Biztonsága – az ML-KEM-hez hasonlóan – az MLWE és a kapcsolódó Module-SIS (Short Integer Solution) problémák nehézségén alapul. A séma a Fiat–Shamir with Aborts keretrendszerre épül.

11.4.1 Paraméterek

Legyen $q = 8380417$, $n = 256$, $R_q = \mathbb{Z}_q[X]/(X^n + 1)$. A biztonsági szinttől függően $(k, \ell) \in \{(4, 4), (6, 5), (8, 7)\}$ (ML-DSA-44, ML-DSA-65, ML-DSA-87). Jelölje S_η az R_q azon elemeinek halmazát, amelyek együtthatói $[-\eta, \eta]$ -ban vannak.

11.4.2 Kulcsgenerálás (KeyGen)

1. Generáljunk egyenletes eloszlású mátrixot $\mathbf{A} \in R_q^{k \times \ell}$ egy véletlenszerű seedből.
2. Mintavételezzünk titkos vektorokat: $\mathbf{s}_1 \leftarrow S_\eta^\ell$, $\mathbf{s}_2 \leftarrow S_\eta^k$.
3. Számoljuk ki:

$$\mathbf{t} = \mathbf{A} \cdot \mathbf{s}_1 + \mathbf{s}_2 \in R_q^k$$

A publikus kulcs: $(\mathbf{A}, \mathbf{t})$. A titkos kulcs: $(\mathbf{s}_1, \mathbf{s}_2)$.

11.4.3 Aláírás (Sign)

Adott egy üzenet μ és a titkos kulcs $(\mathbf{s}_1, \mathbf{s}_2)$:

1. Válasszunk egyenletes eloszlású maszkot: $\mathbf{y} \leftarrow S_{\gamma_1-1}^\ell$.
2. Számoljuk ki az elköteleződést: $\mathbf{w} = \mathbf{A} \cdot \mathbf{y}$.
3. Bontsuk fel $\mathbf{w}$ -t felső és alsó részre: $\mathbf{w}_1 = \text{HighBits}(\mathbf{w})$.
4. Számoljuk ki a kihívást (challenge): $c = H(\mu \| \mathbf{w}_1) \in R_q$, ahol c egy τ darab ± 1 együtthatóval rendelkező polinom.
5. Számoljuk ki a választ:

$$\mathbf{z} = \mathbf{y} + c \cdot \mathbf{s}_1$$

6. **Elutasítás (rejection sampling):** Ha $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ vagy $\|\text{LowBits}(\mathbf{Az} - c\mathbf{t})\|_\infty \geq \gamma_2 - \beta$, ismételjük az 1. lépéstől. (Itt $\beta = \tau \cdot \eta$.)

Az aláírás: $\sigma = (\mathbf{z}, c)$.

11.4.4 Ellenőrzés (Verify)

Adott a publikus kulcs $(\mathbf{A}, \mathbf{t})$, az üzenet μ és az aláírás $(\mathbf{z}, c)$:

1. Ellenőrizzük: $\|\mathbf{z}\|_\infty < \gamma_1 - \beta$.
2. Számoljuk ki:

$$\mathbf{w}'_1 = \text{HighBits}(\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t})$$

3. Ellenőrizzük: $c = H(\mu \| \mathbf{w}'_1)$.

11.4.5 Helyesség

A verifikáció azért működik, mert:

$$\mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t} = \mathbf{A}(\mathbf{y} + c\mathbf{s}_1) - c(\mathbf{A}\mathbf{s}_1 + \mathbf{s}_2) = \mathbf{A}\mathbf{y} - c\mathbf{s}_2 = \mathbf{w} - c\mathbf{s}_2$$

Mivel $c\mathbf{s}_2$ kis normájú, a HighBits művelet ugyanazt az eredményt adja mint az aláíraskor, feltéve hogy az elutasítási feltételek teljesülnek.

11.4.6 Biztonság

Az ML-DSA biztonsága két nehéz problémán alapul:

- **MLWE:** a publikus kulcs $\mathbf{t} = \mathbf{A}\mathbf{s}_1 + \mathbf{s}_2$ nem árulja el $\mathbf{s}_1$ -et és $\mathbf{s}_2$ -t.
- **Module-SIS (MSIS):** rövid vektor $(\mathbf{z}, c)$ találása, amelyre $\mathbf{A}\mathbf{z} - c\mathbf{t}$ adott HighBits értéket ad, számításilag nehéz.

Az elutasítási lépés (rejection sampling) biztosítja, hogy az aláírás $\mathbf{z}$ eloszlása független a titkos kulcstól – ez a Fiat–Shamir with Aborts technika lényege, amelyet Lyubashevsky vezetett be [126].

11.5 SLH-DSA (Stateless Hash-Based Digital Signature Algorithm)

Az SLH-DSA (korábban SPHINCS+) a NIST által 2024-ben szabványosított posztkvantum digitális aláírási séma (FIPS 205) [115]. Az ML-DSA-val ellentétben biztonsága kizárólag hash-függvények tulajdonságain alapul – nem igényel rácsalapú feltételezést. Ez teszi a legkonzervatívabb választássá, mivel a hash-függvények biztonsága jól megértett és évtizedes kriptóanalízis támasztja alá.

11.5.1 Építőelemek

Az SLH-DSA három hash-alapú primitívre épül:

1. WOTS+ (Winternitz One-Time Signature): Egyszer használható aláírási séma. Adott egy titkos kulcs $(sk_0, sk_1, \dots, sk_{\ell-1})$, ahol minden sk_i egy n bájtos véletlenszerű érték. A publikus kulcs:

$$pk_i = F^{w-1}(sk_i), \quad i = 0, \dots, \ell - 1$$

ahol F egy egyirányú hash-lánc függvény és w a Winternitz paraméter (tipikusan $w = 16$). Üzenet M aláírásakor az üzenetet ℓ darab, bázis- w számjegyre $m_0, \dots, m_{\ell-1}$ bontjuk (beleértve egy ellenőrző összeget), és az aláírás:

$$\sigma_i = F^{m_i}(sk_i), \quad i = 0, \dots, \ell - 1$$

Az ellenőrző a lánc hátralévő részét számítja ki:

$$pk'_i = F^{w-1-m_i}(\sigma_i) \stackrel{?}{=} pk_i$$

2. XMSS (eXtended Merkle Signature Scheme): Egy bináris Merkle-fa, amelynek $2^{h'}$ darab levelén WOTS+ publikus kulcsok hash-értékei állnak. A fa gyökere egyetlen rövid publikus kulcsot ad. Aláírásakor az aktuális WOTS+ aláírás mellé csatoljuk a Merkle-fa hitelesítési útvonalát (authentication path): h' darab testvér-csomópont hash-értékét, amelyekből a gyökér rekonstruálható.

3. Hypertree: Egy d rétegű XMSS-fák kaszkádja. A legfelső fa aláírja a következő szint gyökereit, és így tovább. Az összesen $h = d \cdot h'$ szint 2^h egyszer használható aláírást biztosít.

11.5.2 Az SLH-DSA aláírás

Az SLH-DSA állapotmentessé teszi a hypertree-t egy FORS (Forest of Random Subsets) néhányszer használható aláírási séma segítségével:

1. Adott üzenet M . Számoljunk egy véletlenszerű indexet: $\text{idx} = H_{\text{msg}}(R \| pk \| M)$, ahol R egy friss véletlenszerű érték.
2. Az idx kijelöli, hogy a hypertree melyik WOTS+ levele fogja aláírni a FORS publikus kulcsot.
3. **FORS aláírás:** Az üzenet hash-értékét k darab, $\log_2 t$ bites blokkra bontjuk. Minden blokk egy indexet ad meg k darab, egyenként t levelű bináris fában. A FORS aláírás a kijelölt levelek és a hozzájuk tartozó hitelesítési útvonalak, a FORS publikus kulcs pedig a k fa gyökereinek hash-értéke.
4. A FORS publikus kulcsot a hypertree aláírja a kijelölt WOTS+ levéllel és a teljes útvonallal a gyökérig.

Az aláírás tehát: $\sigma = (R, \sigma_{\text{FORS}}, \sigma_{\text{HT}})$.

11.5.3 Ellenőrzés

1. R -ből és az üzenetből újra kiszámoljuk idx -et és az üzenet hash-blokkjait.
2. A σ_{FORS} -ból rekonstruáljuk a FORS publikus kulcsot.
3. A σ_{HT} -ból lépésről lépésre rekonstruáljuk a hypertree gyökerét és ellenőrizzük, hogy egyezik a tárolt publikus kulccsal.

11.5.4 Biztonság

Az SLH-DSA biztonsága három hash-függvény tulajdonságon múlik:

- **Egyirányúság (preimage resistance):** $F(x)$ -ből nem lehet x -et visszanyerni.
- **Második preimage rezisztencia:** Adott x -hez nem található $x' \neq x$ amire $F(x) = F(x')$.
- **Ütközésállóság (collision resistance):** Nem található két különböző bemenet azonos hash-értékkel.

Grover algoritmus [97] a preimage keresést $O(2^{n/2})$ -re gyorsítja a klasszikus $O(2^n)$ -ról, ezért az SLH-DSA nagyobb hash-kimenettel dolgozik: $n \in \{128, 192, 256\}$ bit a NIST biztonsági szinteknek megfelelően. Ez a kompromisszum eredményezi az SLH-DSA fő hátrányát: az aláírás mérete jelentősen nagyobb (kb. 7–49 KB) mint az ML-DSA-é (kb. 2–4 KB).

11.6 McEliece kriptorendszer

A McEliece kriptorendszert Robert McEliece javasolta 1978-ban [116] – ez az egyik legrégebbi posztkvantum kriptográfiai séma, amely közel öt évtized kriptanalízise ellenére sem tört. Biztonsága a véletlenszerű lineáris kód dekódolásának NP-nehézségén alapul. A NIST posztkvantum szabványosítási folyamatának 4. körébe jutott (Classic McEliece néven), de a nagy kulcsméret miatt egyelőre nem került végleges szabványosításra.

11.6.1 Hibajavító kódok háttere

Legyen $\mathcal{C}$ egy $[n, k, d]$ lineáris kód $\mathbb{F}_2$ felett, azaz egy k -dimenziós altér $\mathbb{F}_2^n$ -ben, minimális Hamming-távolsággal d . A kód generátormátrixa $G \in \mathbb{F}_2^{k \times n}$, amellyel bármely $m \in \mathbb{F}_2^k$ üzenet kódolható: $c = m \cdot G$. A kód legfeljebb $t = \lfloor (d-1)/2 \rfloor$ hibát képes javítani.

A McEliece rendszer egy speciális, hatékonyan dekódolható kódot használ – klasszikusan irreducibilis bináris Goppa-kódot –, amelyet véletlenszerűnek álcáz.

11.6.2 Kulcsgenerálás

1. Válasszunk egy t -hibajavító bináris Goppa-kódot $\mathcal{C}$ paraméterekkel $[n, k, \geq 2t+1]$, generátormátrixa $G \in \mathbb{F}_2^{k \times n}$.
2. Generáljunk egy véletlenszerű $k \times k$ méretű invertálható mátrixot $S \in \mathbb{F}_2^{k \times k}$.
3. Generáljunk egy véletlenszerű $n \times n$ méretű permutációs mátrixot $P \in \mathbb{F}_2^{n \times n}$.
4. Számoljuk ki a publikus kulcsot:

$$G' = S \cdot G \cdot P$$

A publikus kulcs: (G', t) . A titkos kulcs: (S, G, P) és a Goppa-kód dekódoló algoritmus.